EXPERIMENT NO: 07

Title: REST API using TypeScript and MongoDB

Aim : To create a REST API using TypeScript and MongoDB and perform CRUD operations.

Tools Required

- Node.js
- MongoDB (Local or Atlas)
- VS Code
- Postman or any API testing tool

Procedure

Step 1: Project Setup

- Create a new Node.js project
- Initialize it using npm
- Install required dependencies (TypeScript, Express, Mongoose)
- Configure TypeScript using a configuration file

```
C:\Users\YASHFEEN RAAVI\Downloads>mkdir ts-mongo-rest-api

C:\Users\YASHFEEN RAAVI\Downloads>cd ts-mongo-rest-api
```

```
C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api>npm init -y
Wrote to C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api\package.json:

{
  "name": "ts-mongo-rest-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

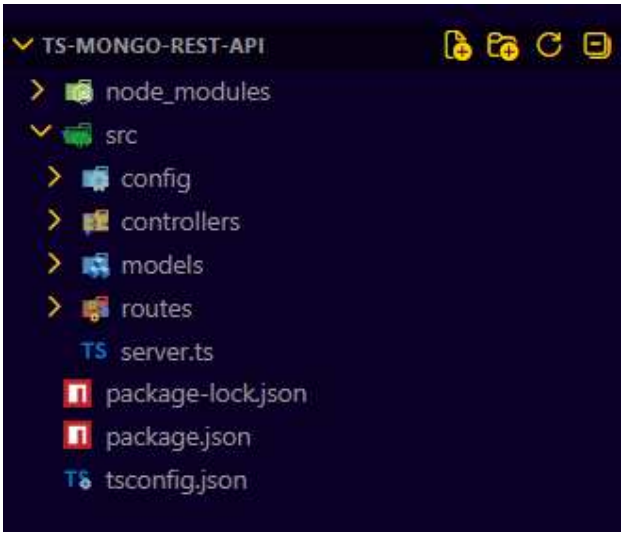
```
C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api>npm install express mongoose dotenv

added 83 packages, and audited 84 packages in 8s

24 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api>npm install --save-dev typescript ts-node nodemon @types/node @types/express

added 56 packages, and audited 140 packages in 9s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api>npx tsc --init

Created a new tsconfig.json

You can learn more at https://aka.ms/tsconfig
```

Edit `tsconfig.json`:

```
{
 "compilerOptions": {
   "target": "ES6",
   "module": "CommonJS",
   "rootDir": "./src",
   "outDir": "./dist",
   "strict": true,
   "esModuleInterop": true
 }
}
```

Create folder structure



Step 2: Create Server

- Create a main server file , src/server.ts

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";
dotenv.config();
const app = express();
app.use(express.json());
mongoose.connect(process.env.MONGO_URI as string)
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.log(err));
app.get("/", (req, res) => {
  res.send("REST API Running...");
});
const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 3 : Create Model : `src/models/User.ts`

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
 name: { type: String, required: true },
 email: { type: String, required: true },
 age: { type: Number, required: true }
});

export default mongoose.model("User", userSchema);
```

Step 4: Create Controller and Implement CRUD Operations
`src/controllers/userController.ts`

```
import { Request, Response } from "express";
import User from "../models/User";
// CREATE
export const createUser = async (req: Request, res: Response) => {
 const user = await User.create(req.body);
 res.status(201).json(user);
};
// READ ALL
export const getUsers = async (req: Request, res: Response) => {
 const users = await User.find();
 res.json(users);
};
// READ BY ID
export const getUserById = async (req: Request, res: Response) => {
 const user = await User.findById(req.params.id);
 res.json(user);
};
// UPDATE
export const updateUser = async (req: Request, res: Response) => {
```

```
 const user = await User.findByIdAndUpdate(req.params.id, req.body, { new:
true });
 res.json(user);
};
// DELETE
export const deleteUser = async (req: Request, res: Response) => {
 await User.findByIdAndDelete(req.params.id);
 res.json({ message: "User deleted" });
};
```

Step 5: Create Routes , `src/routes/userRoutes.ts`

```
import express from "express";

import { createUser,getUsers,getUserById,updateUser,deleteUser } from
"../controllers/userController";

const router = express.Router();

router.post("/users", createUser);

router.get("/users", getUsers);

router.get("/users/:id", getUserById);

router.put("/users/:id", updateUser);

router.delete("/users/:id", deleteUser);

export default router;
```

Step 6: Connect Routes to Server

Update `server.ts`:

```
import userRoutes from "./routes/userRoutes";

app.use("/api", userRoutes);
```

Step 7: Run Project

Add script in `package.json`

"scripts": {

 "dev": "nodemon src/server.ts"

}

Run server:

npm run dev

```
PS C:\Users\YASHFEEN RAAVI\Downloads\ts-mongo-rest-api> npm run dev

> ts-mongo-rest-api@1.0.0 dev
> nodemon src/server.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/server.ts`
◇ injected env (1) from .env // tip: ⌘ enable debugging { debug: true }
Server running on port 5000
MongoDB Connected
```

Step 8 : Testing

Install Thunder Client

1. Open **VS Code**
2. Go to **Extensions (Ctrl + Shift + X)**
3. Search: **Thunder Client**
4. Click **Install**

After install, you'll see a ⚡ Thunder icon in the left sidebar.

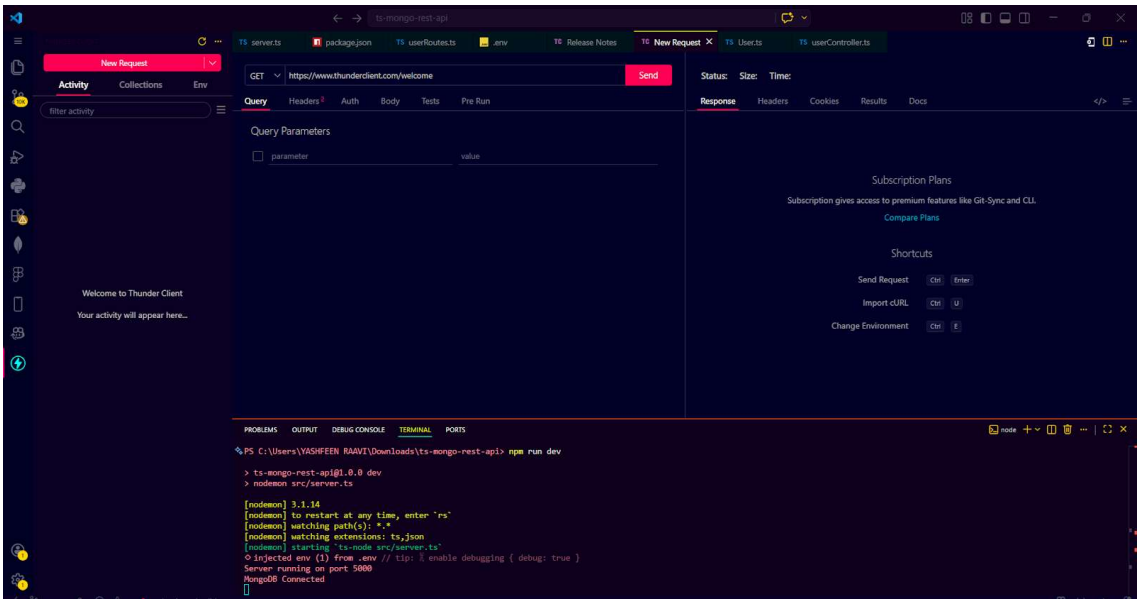
Open Thunder Client

Click the ⚡ icon → then click:

New Request

You'll see:

- Method (GET, POST, PUT, DELETE)
- URL box
- Body tab



Set Base URL

Base URL is: http://localhost:5000/api

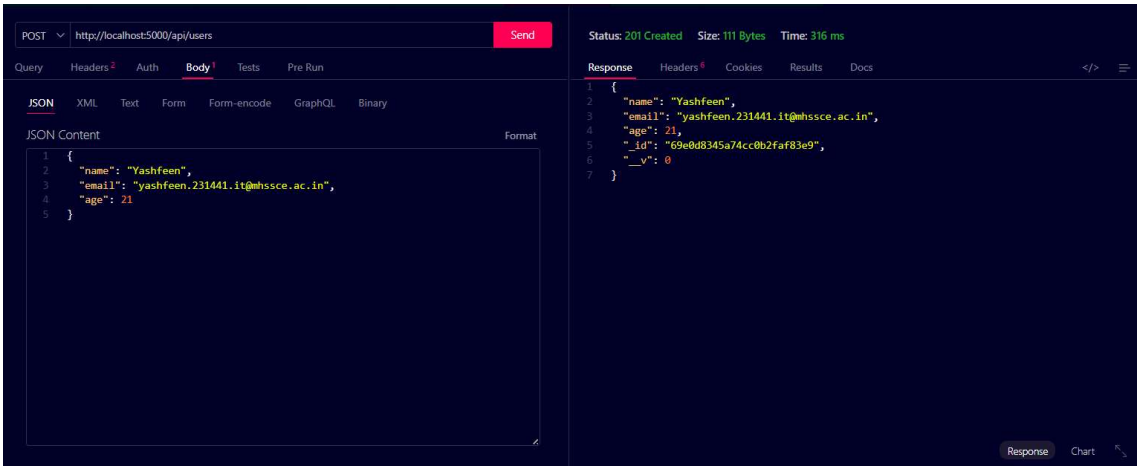
Test CRUD APIs

1. CREATE USER (POST)

Setup:

- **Method: POST**
- **URL:**

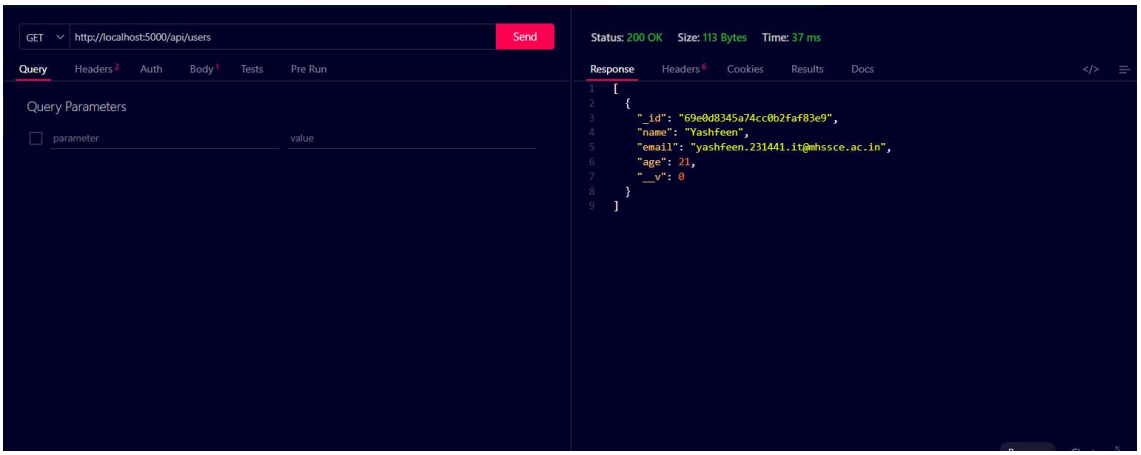
http://localhost:5000/api/users



2. GET ALL USERS

- **Method: GET**
- **URL:**

http://localhost:5000/api/users

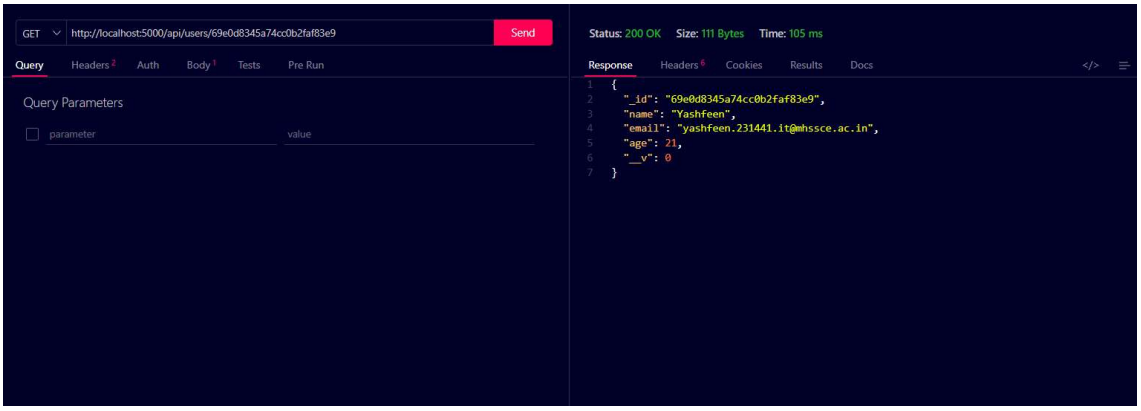


3. GET USER BY ID

- **Method: GET**
- **URL:**

http://localhost:5000/api/users/<USER_ID>

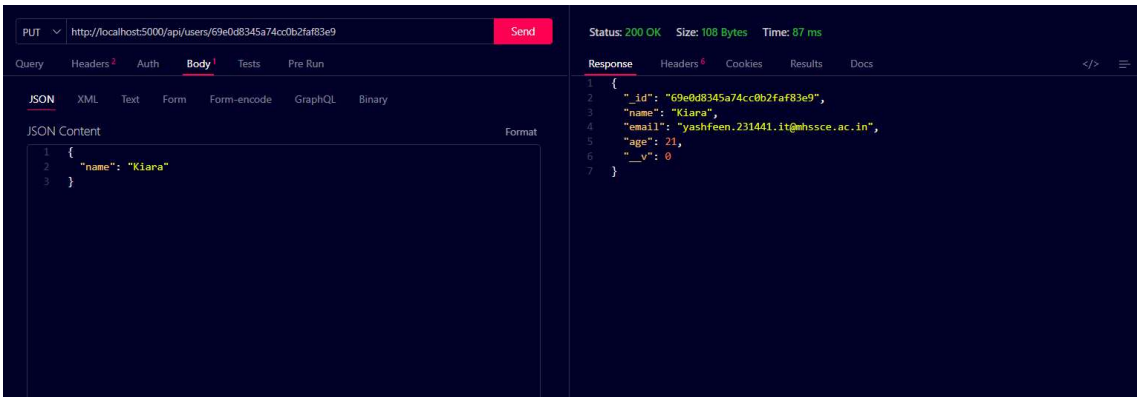
Replace `<USER_ID>` with actual `_id` from MongoDB response



4. UPDATE USER

- **Method: PUT**
- **URL:**

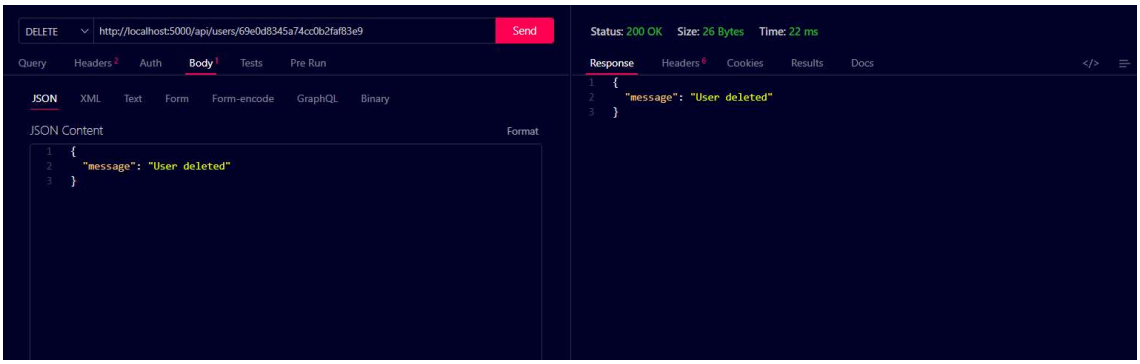
http://localhost:5000/api/users/<USER_ID>



5. DELETE USER

- **Method: DELETE**
- **URL:**

http://localhost:5000/api/users/<USER_ID>



Result

REST API successfully created and tested with all CRUD operations.

Conclusion

The experiment demonstrates how to build a REST API using TypeScript and MongoDB and perform database operations.